Raspberry Pi Onramp

A Beginner’s Guide to Tinkering with Hardware and Code

Abdelbacet Mhamdi

2026-06-20

MT @ ISET Bizerte

Outline

1. General Overview

Why Use Raspberry Pi

Raspberry Pi is a popular choice for beginners because it is affordable, well-documented, and has a large community. It ships with a familiar Debian-based environment, making it easy to get started with Linux. It is equally suited for advanced users who want to build custom hardware projects, run home servers, or experiment with electronics via its 40-pin GPIO header.

What is Raspberry Pi

Raspberry Pi is a **low-cost** operating systems. It is a popular platform for single-board computer *capable of running* Linux-based embedded systems, IoT projects, home automation, education, and prototyping. It was originally developed by the Raspberry Pi Foundation (now Raspberry Pi Ltd.) with the goal of promoting computer science education.

The official operating system is **Raspberry Pi OS** (formerly called Raspbian until May 2020). It is based on the Debian Linux distribution and uses the `apt` (Advanced Package Tool) package manager.

Before installing any software, it is good practice to refresh the local package index and then apply all available upgrades:

```Bash
sudo apt update  # refresh the package list
sudo apt full-upgrade  # upgrade packages, handling dependency changes
```

`full-upgrade` is preferred over `upgrade` because it correctly handles cases where packages need to be added or removed to satisfy updated dependencies.

To install a specific package:

```Bash
sudo apt install git  # install git
```

SSH

Secure Shell Configuration

1.1 SSH Setup

SSH (Secure Shell) is a cryptographic network protocol for secure remote access to a machine over an unsecured network. On Raspberry Pi OS, the `openssh-server` package is already installed but **disabled by default** for security reasons.

There are three common ways to enable it:

Option 1 — raspi-config (recommended for interactive use):

```bash
sudo raspi-config
# Navigate to: Interface Options → SSH → Enable
```

Option 2 — systemctl (enable on a running system):

```bash
sudo systemctl enable ssh  # enable SSH to start on boot
sudo systemctl start ssh  # start the SSH service immediately
```

Option 3 — headless setup (before first boot):

Create an empty file named `ssh` (no extension) in the `/boot` or `/bootfs` partition of the SD card. The OS will detect it on first boot, enable the SSH service, and delete the file.

Once SSH is enabled, connect from another machine with:

```bash
ssh <username>@<hostname>.local
# e.g. ssh pi@raspberrypi.local
```

INFO

Create a new connection profile of type Ethernet on the physical interface enp4s0. The PC will act as a DHCP server and automatically assign an IP address to any device connected via Ethernet (the Pi), while also enabling NAT to share the laptop's internet connection.

```bash
nmcli connection add type ethernet ifname enp4s0 con-name "rpi-share" ipv4.method shared
```

Activate and inspect the connection

```bash
nmcli connection up "rpi-share"
nmcli connection show "rpi-share"
```

Perform a ping sweep across all 254 addresses, without port scanning

```bash
nmap -sn 10.42.0.0/24
```

1.1.1 SSH Key Authentication

Connecting with a password is convenient but less secure. The recommended approach is **public-key authentication**: you generate a key pair on your client machine, then place the public key on the Raspberry Pi. From that point on, no password is needed and brute-force attacks become ineffective.

1.1.2 How it works

A key pair consists of two files:

Private key (`~/.ssh/rpi`) stays on your machine, never shared.
Public key (`~/.ssh/rpi.pub`) copied to the Pi. It can only be used to verify someone who holds the matching private key.

1.1.3 Generate a key pair (on your local machine)

Ed25519 is the current recommended algorithm: it is compact, fast, and more secure than RSA.

```bash
ssh-keygen -t ed25519 -C "your_email@example.com"
```

You will be prompted for:

File location press Enter to accept the default (`~/.ssh/rpi`).

Passphrase strongly recommended. It encrypts the private key on disk so that even if the file is stolen, it cannot be used without the passphrase.

The command produces two files:

```
~/.ssh/rpi      # private key — keep this secret
~/.ssh/rpi.pub  # public key  — this goes on the Pi
```

1.1.4 Copy the public key to the Raspberry Pi

The simplest method is `ssh-copy-id`, which handles directory creation and permissions automatically:

```bash
ssh-copy-id <username>@<pi-ip-address>
# e.g. ssh-copy-id pi@10.42.0.39
```

You will be asked for your Pi password one final time. After this step, the public key is appended to `~/.ssh/authorized_keys` on the Pi.

1.1.5 Connect using the key

```bash
ssh <username>@<pi-ip-address>
# e.g. ssh pi@192.168.1.123
```

SSH automatically finds and uses `~/.ssh/id_ed25519`. If you set a passphrase, you will be prompted for it once. To avoid re-entering it every session, add the key to the SSH agent:

```bash
eval "$(ssh-agent -s)"   # start the agent
ssh-add ~/.ssh/rpi  # load the key (prompts for passphrase once)
```

INFO

SSH Client Configuration File

The `~/.ssh/config` file lets you define **host aliases** with all connection parameters, so you never have to remember long `ssh` commands again.

```
1 Host rpi
2     HostName raspberrypi.local
3     User pi
4     IdentityFile ~/.ssh/rpi
5     ServerAliveInterval 60
```

With this configuration in `~/.ssh/config`, you can simply run:

```bash
ssh rpi
```

Bash

Once key authentication is confirmed to be working, you can disable password logins entirely to harden the Pi against brute-force attacks. Edit the SSH server config:

```bash
sudo vim /etc/ssh/sshd_config
```

Find and set the following lines *(restart the ssh service afterwards)*:

```
PasswordAuthentication no
PubkeyAuthentication yes
```

Then restart the SSH service.

WARNING

⚠ Do not disable password login until you have verified that key authentication works in a separate terminal session. Locking yourself out would require physical access to the Pi to recover.

Git and GitHub

Version Control and Collaboration

Git is a distributed version control system that tracks changes to files and directories over time. It allows you to commit snapshots of your work, branch off for experiments, and merge changes back together.

GitHub is a cloud-based hosting platform for Git repositories. It adds collaboration features such as pull requests, issues, and Actions on top of standard Git.

To install Git:

```bash
sudo apt install git
```

Basic initial configuration:

```Bash
git config --global user.name  "Your Name"
git config --global user.email "you@example.com"
```

To clone an existing repository from GitHub:

```Bash
git clone https://github.com/a-mhamdi/rpi-onramp.git
```

Using a key with GitHub

The same key pair `rpi` and `rpi.pub` can authenticate you to GitHub, removing the need for HTTPS tokens when pushing code.

Display your public key and copy the output. Then go to `GitHub` → `Settings` → `SSH and GPG keys` → `New SSH key`, paste it in, and save. Test the connection:

```bash
ssh -T git@github.com
# Expected: Hi <username>! You've successfully authenticated...
```

To switch an existing local repository from HTTPS to SSH:

```bash
git remote set-url origin git@github.com:<user>/<repo>.git
```

Lazygit

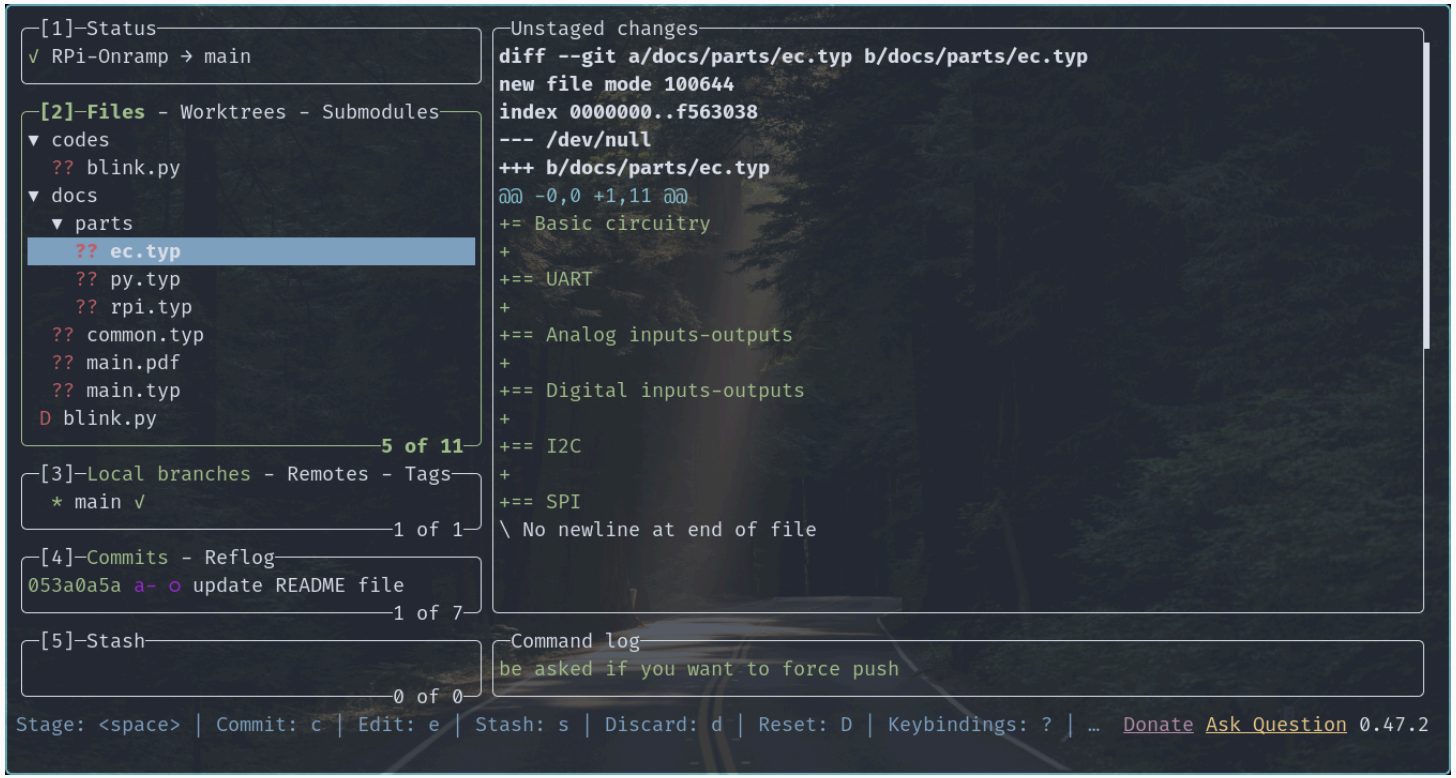
Lazygit is a terminal UI for Git that makes complex operations simple — stage individual lines, perform interactive rebases, and cherry-pick commits, all without memorising obscure flags. It runs directly in your terminal, so there is no need to switch between your editor and a separate GUI application. It is especially useful on the Raspberry Pi where you are often working entirely over SSH with no desktop environment.

To install it on Raspberry Pi OS:

```bash
sudo apt install lazygit
```

Launch it from inside any Git repository by simply running `lazygit`. Press `?` at any time to view the full list of keybindings.

1.2 Git and GitHub



For full documentation and source code, see github.com/jesseduffield/lazygit.

2. Julia Codes

Julia is a high-level, high-performance programming language designed for numerical and scientific computing. While it is often associated with workstations and servers, Julia runs well on ARM-based hardware including the **Raspberry Pi**. This makes it an attractive choice for edge computing, data logging, sensor fusion, and lightweight network services running directly on embedded Linux systems.

This document covers how to install Julia on a **Raspberry Pi**, how to manage GPIO and serial connections, how to use Julia's networking stack, and how to write practical programs that tie these capabilities together.

Getting Started with Julia on Raspberry Pi

Installation and First Steps

2.1.1 Hardware Requirements

Julia can run on any Raspberry Pi model that supports a 64-bit ARM operating system, starting from the Raspberry Pi 3. The Raspberry Pi 4 or 5 is recommended for interactive use and package compilation due to its faster CPU and larger RAM. A microSD card of at least 16 GB is advisable, as Julia's package depot and precompiled artifacts can occupy several gigabytes.

2.1.2 Operating System

Install the 64-bit version of Raspberry Pi OS (formerly Raspbian). Julia's official ARM builds target `aarch64`, so the 64-bit OS is required to use them directly. You can download the image from the Raspberry Pi website and flash it with Raspberry Pi Imager.

After first boot, run a full system update before installing Julia:

```bash
sudo apt update && sudo apt upgrade -y
```

2.1.3 Installing Julia

The recommended way to install Julia on Raspberry Pi is through `juliaup`, the official Julia version manager.

```Bash
curl -fsSL https://install.julialang.org | sh
```

Follow the prompts. After installation, open a new shell session or source your profile:

```Bash
source ~/.bashrc
```

Verify the installation:

```Bash
julia --version
```

`juliaup` allows you to maintain multiple Julia versions and switch between them with

```Bash
juliaup default <version>
```

INFO

If you prefer a simpler path without `juliaup`, the Raspberry Pi OS repositories include a Julia package, though it may be an older version:

```Bash
sudo apt install julia
```

Using the Julia REPL

Interactive Programming and Package Management

Start Julia by typing `julia` in the terminal. You will be greeted by the interactive REPL *(Read-Eval-Print Loop)*. From here you can enter expressions, manage packages, and run scripts.

Julia has four REPL modes:

Julia mode *(default)* execute Julia expressions.
Package mode *(press])* manage packages with `add`, `rm`, `status`, `update`.
Help mode *(press ?)* display documentation for any symbol.
Shell mode *(press ;)* run shell commands without leaving Julia.

2.2.1 Package Depot and Precompilation

To install a package, enter package mode and type:

```
1  ] add Plots
```
Julia

To exit the REPL:

```
1  exit()
```
Julia

Julia compiles packages on first use. On a Raspberry Pi this can be slow — several minutes for large packages — but subsequent launches are fast. Precompilation happens automatically when you `add` a package or `using` it for the first time.

2.2.2 Julia Onramp

> </> Source code is available at [codes/julia-onramp.ipynb]

2.2.3 Application: Basic Backprop

- An implementation of backpropagation for a simple perceptron.
- Gradients computation and weights update using Julia's basic operations.

> </> Source code is available at codes/basic-backprop.jl

3. Basic Circuitry

Digital I/O

Interfacing with the Physical World

The Raspberry Pi exposes **40 GPIO pins** *(on models B+ and later)*, each configurable as either **input** or **output** at runtime.

Output mode
- Drive pin HIGH (3.3 V) or LOW (0 V)
- Power LEDs, relays, transistors
- Max current: **16 mA** per pin

Input mode
- Read external logic levels
- Optional pull-up / pull-down resistors
- Voltage-safe range: 0 – 3.3 V

⚠ **WARNING**

GPIO operates at 3.3 V logic — never connect 5 V signals directly.

Julia accesses GPIO through the **PiGPIO.jl** package, which wraps the `pigpio` C daemon running on the Pi.

> </> Source code is available at codes/digital-io.jl

> ✎ **NOTE**
>
> Requires `pigpiod` running: `sudo pigpiod` · Install: `]add PiGPIO`

Analog I/O

Measuring and Controlling Continuous Signals

3.2 Analog Inputs-outputs

Unlike Arduino, the Raspberry Pi has **no built-in ADC** — all GPIO pins are strictly digital. Analog I/O requires external hardware.

Analog Input (ADC)

- Common chip: **MCP3008** (10-bit, 8-ch)
- Communicates over **SPI**
- Resolution: 1024 steps over 0 – 3.3 V
- 3.2 mV per step

Analog Output (DAC)

- Common chip: **MCP4725** (12-bit)
- Communicates over **I²C**
- Resolution: 4096 steps over 0 – 3.3 V
- 0.8 mV per step

`PiGPIO.jl` provides hardware-timed PWM on any GPIO pin via the `pigpiod` daemon, giving clean pulses without CPU spin.

> **INFO**
>
> PWM can approximate analog output without a DAC — useful for motor speed and LED dimming.

</> Source code is available at [codes/analog-io.jl]

Line Encoding Schemes

NRZ-L · NRZ-I · Manchester · Differential Manchester

3.3 Line Encoding

Line encoding is how binary data (0s and 1s) gets mapped to a **physical signal** on a wire — defined by voltage levels and when they change.

Voltage Level
High (+V) or Low (–V / 0V) represents a bit

Transitions
A signal change (edge) can itself carry meaning

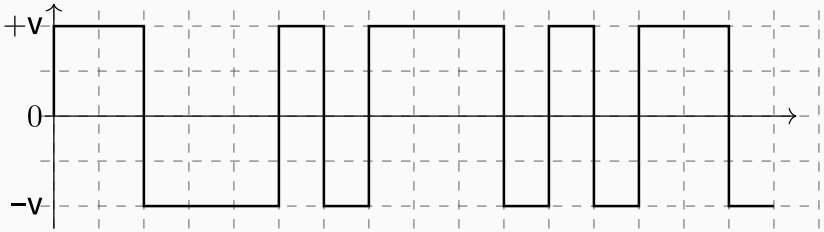
Clock Recovery
Receiver needs to stay in sync with the sender

Different schemes trade off between **simplicity**, **bandwidth**, and **self-clocking**.

NRZ-L — Non-Return to Zero Level

Rule: Level directly represents the bit value.

- **High** (+V) → bit **0**
- **Low** (-V) → bit **1**
- Signal stays at that level for the entire bit period
- No transition needed — hence “non-return to zero”



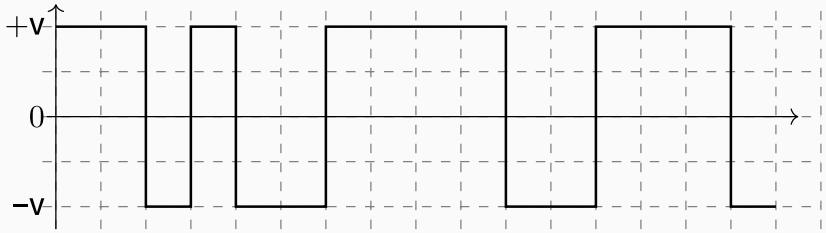
Advantage: Simple, efficient use of bandwidth. (Used in RS-232)

Problem: Long runs of the same bit produce no transitions → receiver loses clock sync.

NRZ-I — Non-Return to Zero Inverted

Rule: A **transition** encodes the bit, not the level.

- Bit **1** → **transition** at the start of the bit period
- Bit **0** → **no transition** (level stays the same)



This is **differential** coding: the meaning depends on change, not absolute voltage.

Advantage: Immune to polarity inversion (swapped wires don't corrupt data). (Used in USB, HDLC)

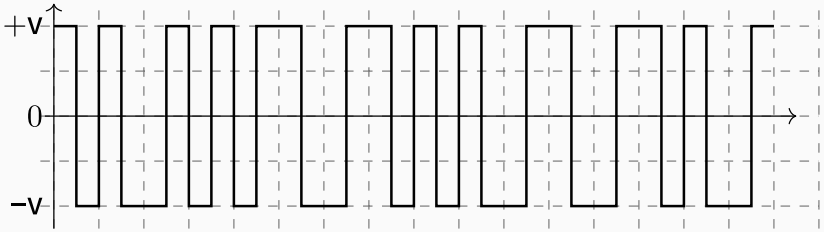
Problem: Long runs of **0s** still produce no transitions → still loses sync.

Manchester Encoding

Rule: Each bit period is split in half — the bit is encoded by the **mid-bit transition direction**.

- Bit **0** → **High→Low** transition at mid-bit
- Bit **1** → **Low→High** transition at mid-bit

Every bit has a transition in the middle → **self-clocking**: receiver can always recover the clock.



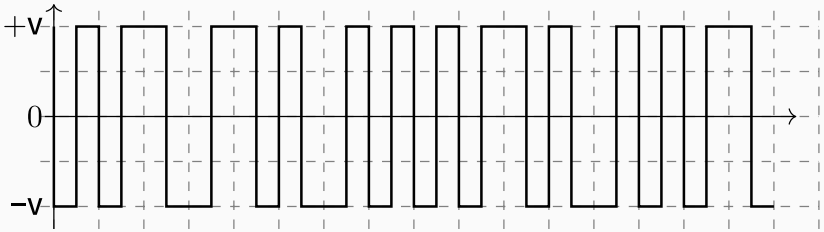
Advantage: No long runs without transitions, no DC component. (Used in 10BASE-T Ethernet, IR remotes)

Cost: Requires **2× the bandwidth** of NRZ — each bit takes two signal intervals.

Differential Manchester Encoding

Rule: Mid-bit transition is **always** present *(for clocking)*.
The bit value is encoded by what happens **at the start**:

- Bit **0** → **transition** at the start of the bit period
- Bit **1** → **no transition** at the start



Combines ideas from NRZI *(differential)* and Manchester *(guaranteed mid-bit edge)*.

Advantages: Always self-clocking (mid-bit edge). Immune to polarity inversion. (Used in Token Ring, IEEE 802.5)

Cost: Same 2× bandwidth overhead as Manchester.

4B/5B — Block Encoding

Rule: Every group of **4 data bits** is replaced by a **5-bit code word** chosen to guarantee enough transitions.

The 5-bit codes are picked so no code has more than **one leading zero** or **two trailing zeros** — ensuring NRZI transitions happen frequently.

> **INFO**
>
> 4B/5B is always paired with NRZI — it's a pre-coding step, not a standalone scheme.

```
4-bit   5-bit
──────  ──────
 0000   11110
 0001   01001
 0010   10100
  ...
 1111   11101

Unused codes → control symbols (idle, start, end)
```

Summary Comparison

Scheme	Self-Clocking	DC-Free	BW Overhead	Used In
NRZ-L	×	×	0%	RS-232
NRZ-I	Partial	×	0%	USB, HDLC
Manchester	✓	✓	100%	10BASE-T, IR
Diff. Manchester	✓	✓	100%	Token Ring
4B/5B + NRZI	✓	Partial	25%	Fast Ethernet

UART and RS-232

Asynchronous Serial Communication Protocols

What is UART?

UART — Universal Asynchronous Receiver/Transmitter — is a hardware protocol for serial communication between two devices.

Asynchronous
No shared clock line. Both sides agree on speed in advance.

Serial
Bits sent one at a time over a single wire per direction.

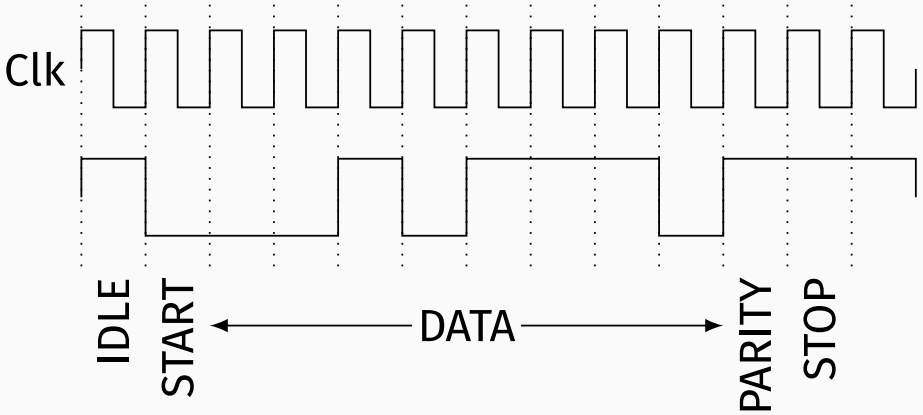
Full-Duplex
TX and RX are separate wires — both sides can talk at once.

UART is a **logic-level protocol**. RS-232 is one electrical standard built on top of it — defining voltages, connectors, and cable lengths.

UART Frame Structure

A UART frame wraps each byte with control bits:

Idle line sits **High** when nothing is sent
Start bit always **0** (Low), signals frame start
Data bits 5 to 9 bits, LSB first (typically 8)
Parity bit optional error check (even/odd/none)
Stop bit(s) always **1** (High), 1 or 2 bits



The receiver detects the **High→Low** edge of the start bit, then samples each bit at the center of its period.

Baud Rate

Baud rate = number of **symbols per second** on the line. For UART (one bit per symbol): baud rate = bit rate.

Both sides must be configured to the **same baud rate** — there is no negotiation. A mismatch causes garbage data.

Common standard rates:

Baud Rate	Bit Period
9 600	104.2 µs
19 200	52.1 µs
115 200	8.68 µs
921 600	1.08 µs

UART **tolerates** a clock mismatch of up to **±2–3%** before bit sampling drifts enough to cause errors. Over a 10-bit frame, even a 2% error shifts the sample point by **0.2 bit periods** by the last bit.

Parity — Simple Error Detection

The optional **parity bit** is set so the total number of 1-bits in the frame (data + parity) satisfies a rule:

Even parity total count of 1s is even
Odd parity total count of 1s is odd
None no parity bit, faster transmission

Detects: any **single-bit** error.

Misses: any **even number** of flipped bits.

```
Data: 0b10110010
1-bits: 4  (even)

Even parity bit → 0
(total 1s stays 4)

Odd parity bit  → 1
(total 1s = 5)
```

Parity does **not** correct errors — only flags them. For correction, higher-level protocols are needed.

RS-232 — Voltage Levels

RS-232 uses **inverted** and **wide-swing** voltages compared to logic levels:

Logic 1 (Mark / Idle)

Voltage between **−3 V and −15 V**
Typically **−12 V** in practice

Logic 0 (Space / Start)

Voltage between **+3 V and +15 V**
Typically **+12 V** in practice

The **±3 V dead zone** (−3 V to +3 V) is undefined — signals in this range are ignored, giving noise immunity.

Key point: Logic 1 = negative voltage. This is **opposite** to TTL/CMOS logic (where 1 = High). A **level shifter** *(e.g. MAX232)* is required to interface RS-232 with a microcontroller.

RS-232 — DB9 Connector & Signals

The classic RS-232 connector is the **DE-9** (often called DB9).

Pin	Name	Function
1	DCD	Data Carrier Detect
2	RXD	Receive Data
3	TXD	Transmit Data
4	DTR	Data Terminal Ready
5	GND	Signal Ground
6	DSR	Data Set Ready
7	RTS	Request To Send
8	CTS	Clear To Send
9	RI	Ring Indicator

For a minimal connection *(no flow control)*, only **3 wires** are needed:

RTS/CTS add hardware flow control — the sender checks CTS before transmitting to avoid overflowing the receiver.

UART vs RS-232 — What's the Difference?

Aspect	UART	RS-232
What it is	Logic protocol	Electrical standard
Voltage (logic 1)	+3.3 V or +5 V	−3 V to −15 V
Voltage (logic 0)	0 V	+3 V to +15 V
Max cable length	Short (PCB traces)	15 m at 9600 baud
Noise immunity	Low	High (wide swing)
Connector	None defined	DE-9 / DE-25
Interface chip	None needed	MAX232 or similar

✦ UART logic levels are for chip-to-chip communication on a PCB.
✦ RS-232 was designed for long cables in industrial environments.

</> Source code is available at codes/tx-rx.jl

Inter-integrated Circuits

A Multi-Master, Multi-Slave, Packet-Switched, Single-Ended, Serial Communication Bus

What is I²C?

I²C is a **synchronous**, **half-duplex**, **multi-master** serial bus developed by Philips in 1982. Only **2 wires** are needed regardless of how many devices are on the bus.

2 Wires Only
SDA (data) and SCL (clock) — shared by all devices on the bus.

Addressed
Every slave has a 7-bit (or 10-bit) address. No CS pins needed.

ACK/NACK
Slave acknowledges every byte — master knows if data arrived.

Multi-Master
Multiple masters can share the bus with built-in arbitration.

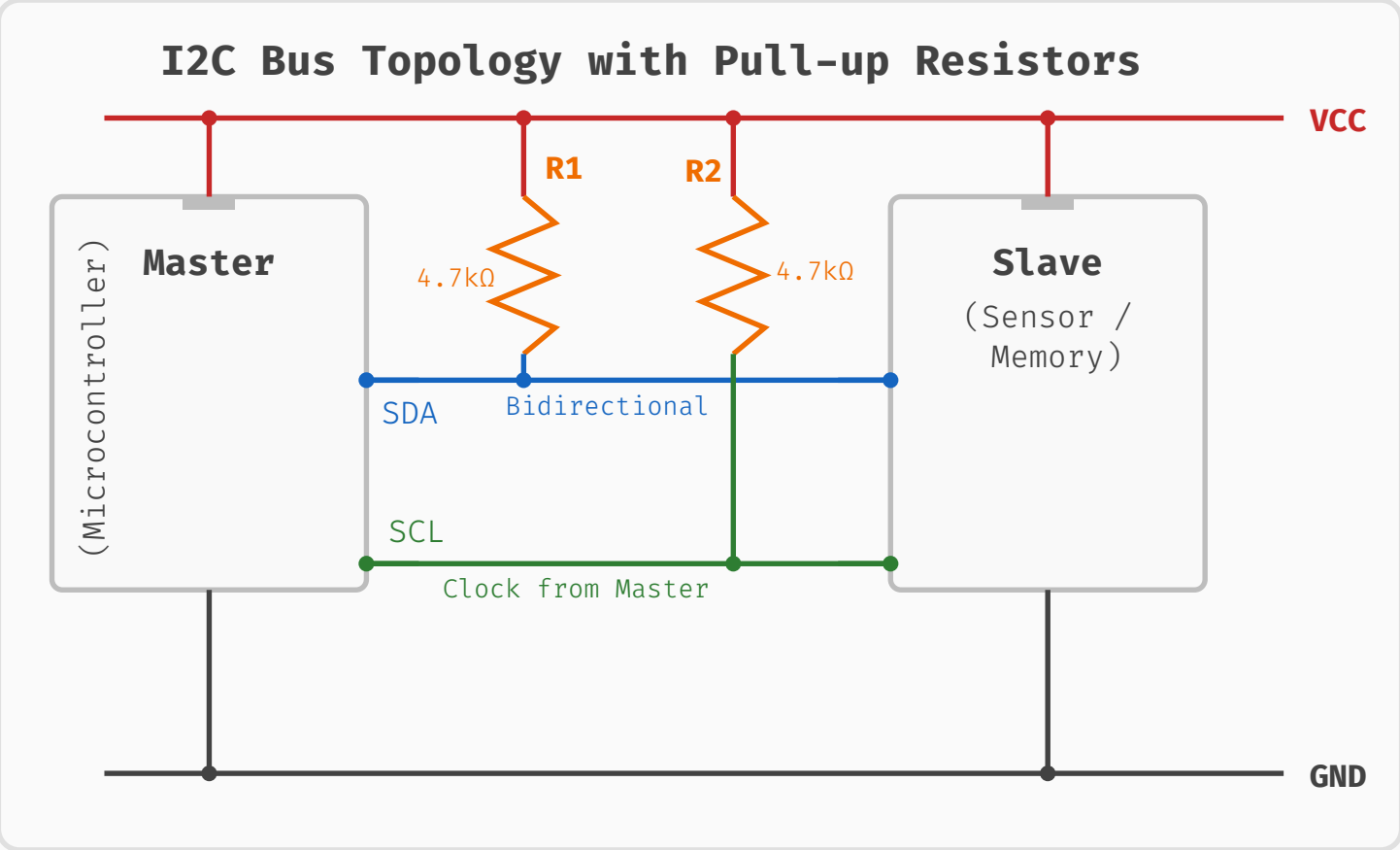
Common uses: sensors (temp, IMU, humidity), EEPROMs, RTCs, LCD controllers, power management ICs.

Bus Topology & Pull-up Resistors

All devices share the same SDA and SCL wires. SDA and SCL are **open-drain** lines — devices can only pull them **Low**. Pull-up resistors connected to VCC bring the lines **High** when nobody is pulling them down.

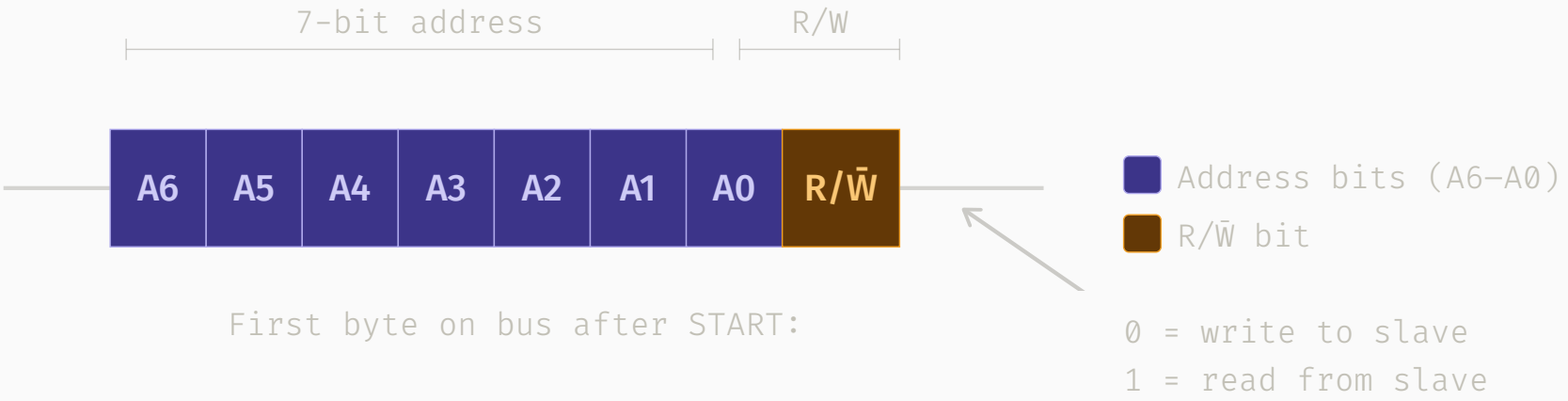
This allows any device to signal without short-circuiting others.

- **Too high** → slow rise time, signal corruption at high speed.
- **Too low** → excessive current, devices struggle to pull Low.


I2C Bus Topology with Pull-up Resistors
VCC
R1
R2
4.7kΩ
4.7kΩ
Master
(Microcontroller)
Slave
(Sensor / Memory)
SDA
Bidirectional
SCL
Clock from Master
GND

Device Addressing

Every slave has a **7-bit address** (most common) hardwired or configurable via address pins.

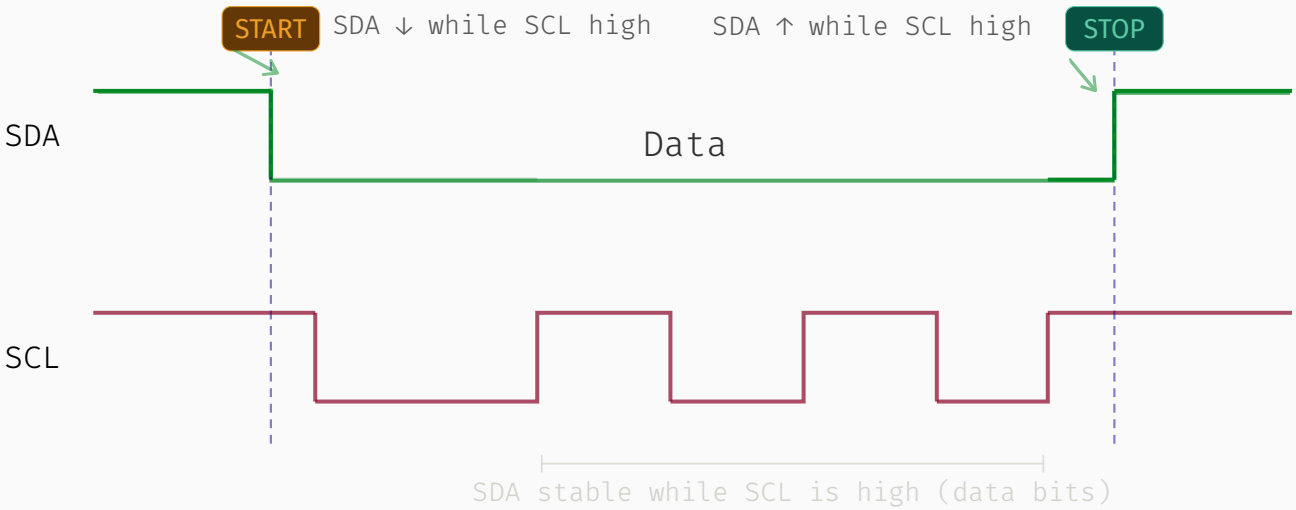


The master sends the address at the start of every transaction, followed by a **R/W̄ bit**:

- **0** → master wants to **write** to slave
- **1** → master wants to **read** from slave

START and STOP Conditions

I²C uses special **bus conditions** — not regular data bits — to delimit transactions. They are unique because SDA changes **while SCL is High** (normally forbidden during data transfer).



START (S)

SDA goes **High→Low** while SCL is **High**.

Signals the beginning of a transaction.

Bus is now “busy”.

STOP (P)

SDA goes **Low→High** while SCL is **High**.

Signals end of transaction.

Bus returns to “idle”.

Data Transfer & ACK/NACK

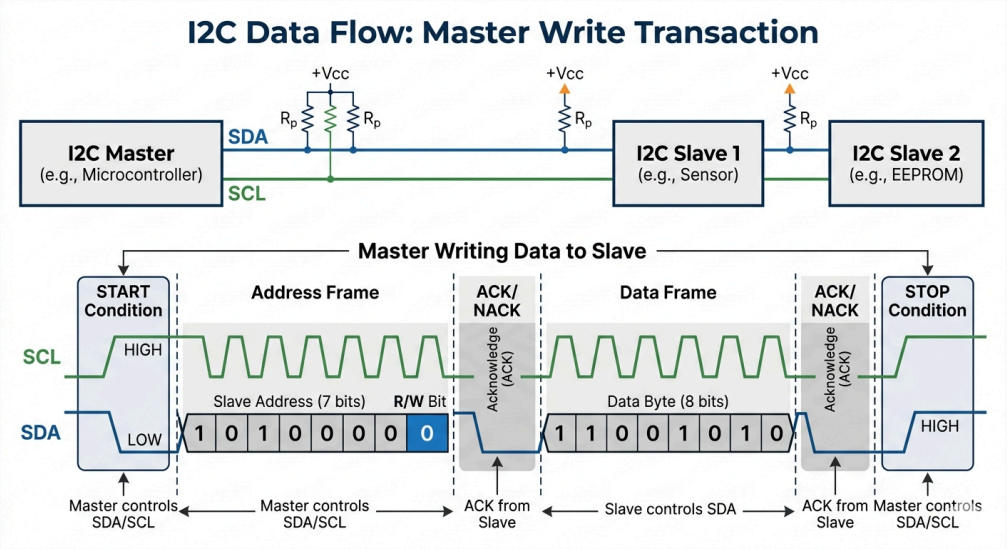
Data is transferred **8 bits at a time**, MSB first. After each byte, there is a mandatory **9th clock pulse** for the acknowledgement bit:

ACK (0) receiver pulls SDA **Low**: “got it, send more”
NACK (1) SDA stays **High**: “stop” or “not ready”

Who sends ACK?
Write transaction slave ACKs each byte from master
Read transaction master ACKs each byte from slave; master sends NACK on the **last** byte to signal it’s done

Write Transaction

A typical register read *(e.g. from a sensor)* uses a **Repeated START**:



Phase 1 — Write

Master sends slave address + register it wants to read from.

Repeated START

Bus stays busy. Master switches direction without a STOP.

Phase 2 — Read

Master re-addresses slave in Read mode. Slave drives SDA.

Clock Stretching

A slave that needs more time to process data can **hold SCL Low** after the master releases it — effectively pausing the clock.

The master, before driving SCL High again, checks if the line is already High. If it finds SCL still Low (slave holding it), it waits until the slave releases it.

Use cases:

- Slow ADC completing a conversion
- EEPROM finishing an internal write cycle
- Slave needing time to prepare the next data byte

Risk: A buggy slave that never releases SCL will **hang the bus**. Some masters implement a timeout and reset.

IMPORTANT

Only possible because SCL is open-drain — slave can pull Low without fighting the master.

I²C Speed Modes

Mode	Max Speed	Pull-up	Notes
Standard Mode (Sm)	100 kHz	4.7 kΩ	Original spec, most devices support
Fast Mode (Fm)	400 kHz	2.2 kΩ	Very widely supported
Fast-Mode Plus (Fm+)	1 MHz	1 kΩ	Requires stronger drivers
High-Speed Mode (Hs)	3.4 MHz	special	Special master code, rare
Ultra-Fast Mode (UFm)	5 MHz	N/A	Unidirectional, no ACK, very rare

INFO

i Most embedded projects use **Standard** or **Fast** mode. Not all slaves support Fast-Mode Plus or above — always check the datasheet. Higher speeds require shorter bus lengths and lower pull-up values.

I²C — Pros & Cons

- **Only 2 wires** regardless of slave count
- Built-in **ACK/NACK** — detects missing slaves
- **Multi-master** with hardware arbitration
- **Clock stretching** for slow slaves
- Simple board routing, long bus possible at low speed

- **Half-duplex** — can't read and write simultaneously
- **Slower** than SPI (max 5 MHz vs 100 MHz)
- **Address conflicts** possible with fixed-address slaves
- Open-drain + pull-ups add **capacitance** — limits bus length at high speed
- Protocol overhead (address + ACK per byte)

</> Source code is available at codes/i2c.jl

Serial Peripheral Interface

A High-Speed Synchronous Communication Protocol

What is SPI?

SPI is a **synchronous**, **full-duplex**, **master-slave** serial bus. Unlike UART, a shared **clock line** is provided by the master — no baud rate agreement needed.

Synchronous
Clock driven by master. Both sides latch on the same edge.

Full-Duplex
Data flows in both directions simultaneously on separate wires.

Master-Slave
Master controls the clock and selects which slave talks.

No Addressing
Slaves are selected by a dedicated chip select (CS) line.

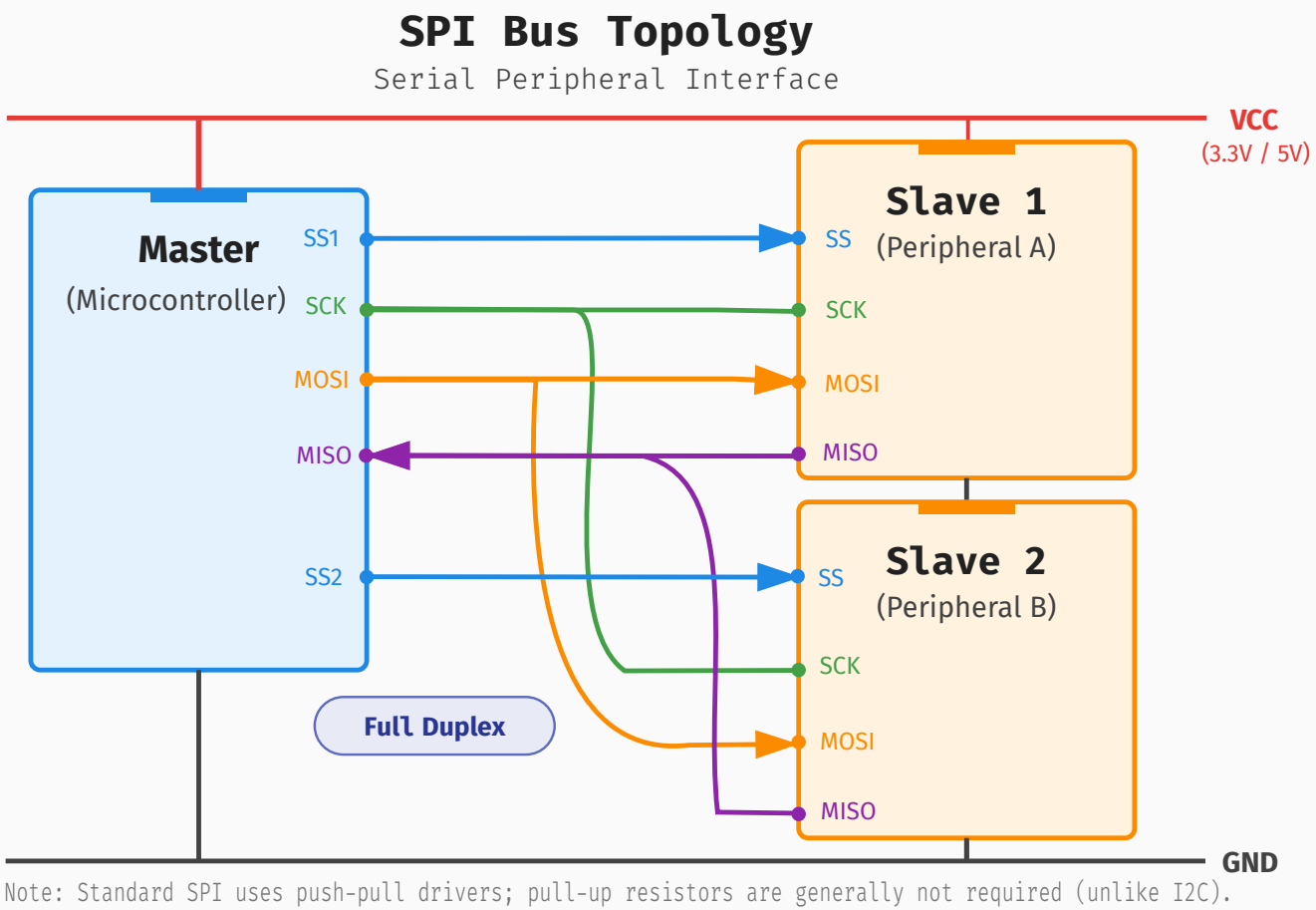
Common uses: flash memory, SD cards, ADCs/DACs, displays, sensors.

SPI Signal Lines

SPI uses **4 wires** (plus ground):

Signal	Dir	Description
SCLK	M→S	Serial Clock — generated by master
MOSI	M→S	Master Out Slave In — data to slave
MISO	S→M	Master In Slave Out — data from slave
CS/SS	M→S	Chip Select — **active low**, selects slave

✦ Some datasheets use SDI/SDO or DIN/DOUT instead of MOSI/MISO.
✦ CS is sometimes called SS (Slave Select) or NSS (active low implied).



Note: Standard SPI uses push-pull drivers; pull-up resistors are generally not required (unlike I2C).

How a Transfer Works

1. Master pulls **CS Low** to select the slave
2. Master generates **clock pulses** on SCLK
3. On each clock cycle:

- Master shifts one bit out on **MOSI**
- Slave shifts one bit out on **MISO**

1. After all bits, master releases **CS High**

Both devices shift simultaneously through an internal shift register it's essentially a **circular exchange** of bytes.

NOTE

Even if you only want to **read**, you must still clock out dummy bytes on MOSI to generate the clock.

SPI Clock Modes (CPOL & CPHA)

Two parameters control timing. Each has 2 options → **4 modes** total.

CPOL — Clock Polarity

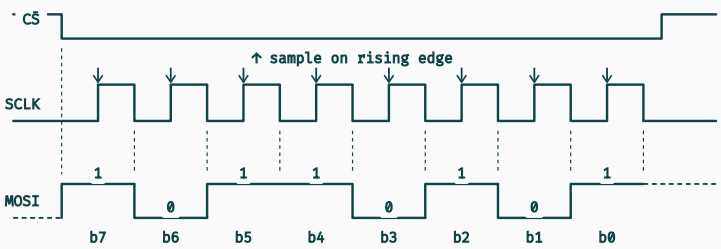
CPOL=0 → clock idles **Low**
CPOL=1 → clock idles **High**

CPHA — Clock Phase

CPHA=0 → data sampled on **leading** edge
CPHA=1 → data sampled on **trailing** edge

Mode	CPOL	CPHA	Idle Clock	Sample On	Common Use
0	0	0	Low	Rising edge	Most sensors, SD cards
1	0	1	Low	Falling edge	Some ADCs
2	1	0	High	Falling edge	Some flash chips
3	1	1	High	Rising edge	Some displays

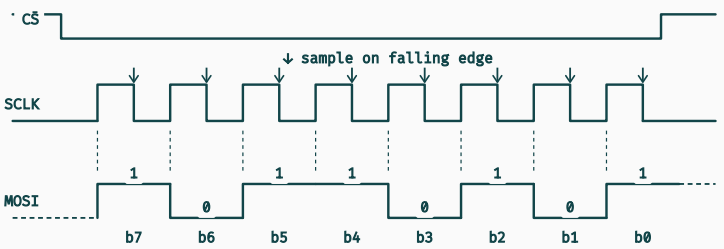
SPI Mode 0 — CPOL=0, CPHA=0



⟶ Sample (rising edge) ----- Drive next bit (falling edge / CS↓) ·· Hi-Z

CPOL = 0 → clock idles low
CPHA = 0 → first bit driven on CS↓ (before SCLK);
subsequent bits on SCLK↓; sampled on SCLK↑

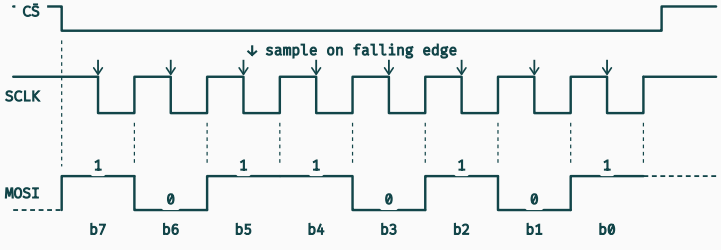
SPI Mode 1 — CPOL=0, CPHA=1



⟶ Sample (falling edge) ----- Drive next bit (rising edge) ----- Hi-Z

CPOL = 0 → clock idles low
CPHA = 1 → first bit driven on SCLK↑;
data sampled on SCLK↓ (second edge)

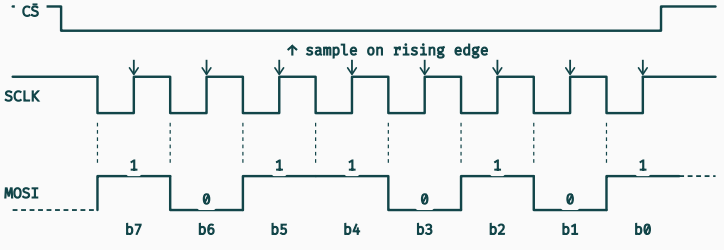
SPI Mode 2 — CPOL=1, CPHA=0



⟶ Sample (falling edge) ----- Drive next bit (rising edge / CS↓) ·· Hi-Z

CPOL = 1 → clock idles high
CPHA = 0 → first bit driven on CS↓;
data sampled on SCLK↓ (first edge); shifted on SCLK↑

SPI Mode 3 — CPOL=1, CPHA=1



⟶ Sample (rising edge) ----- Drive next bit (falling edge) ----- Hi-Z

CPOL = 1 → clock idles high
CPHA = 1 → first bit driven on SCLK↓;
data sampled on SCLK↑ (second edge)

Multiple Slaves

Independent CS lines *(most common)*

Each slave has its own CS pin on the master. SCLK, MOSI, MISO are shared. Only one CS is pulled low at a time.

Pro: Simple, fast, independent.
Con: One CS pin per slave on master.

Daisy-chain *(less common)*

Slaves are chained: MISO of one feeds MOSI of the next. A single CS activates all. Data ripples through the chain.

Pro: Only 1 CS line needed.
Con: All slaves must support it;
latency increases with chain length.

SPI — Pros & Cons

- **Fast** — few MHz to 100 MHz, no protocol overhead
- **Simple** hardware and firmware
- **Full-duplex** — read and write simultaneously
- No addressing logic — CS line handles selection
- Works with almost any slave device

- **4 wires minimum** — grows with slave count (CS pins)
- No acknowledgement — master can't detect absent slave
- No built-in error detection (no parity, no CRC in protocol)
- Short distances only — no defined max cable length
- No multi-master support in standard spec

</> Source code is available at codes/spi.jl

SPI vs UART vs I²C

Feature	SPI	UART	I²C
Clock	Shared (master)	None (async)	Shared (master)
Wires (2 nodes)	4	2	2
Duplex	Full	Full	Half
Speed	Up to 100 MHz	Up to 1 Mbps	Up to 5 Mbps (FM+)
Multi-slave	CS per slave	Point-to-point	Address per slave
Addressing	Hardware CS	None	7/10-bit address
Hardware cost	Low	Very low	Low
Complexity	Low	Very low	Medium

4. Use Case

Anomaly Detection

What and Why?

4.1 What is Anomaly Detection?

INFO

Anomaly detection is the process of identifying patterns, data points, or observations that deviate significantly from expected behaviour in a dataset or data stream.

In the context of sensor data:

A sensor reading is anomalous when it falls outside the statistical or physical envelope of normal operation — whether a single spike, a slow drift, or a structural change in the signal's distribution.

IMPORTANT

Why does it matter?

Early fault detection catch equipment degradation before catastrophic failure
Safety trigger alerts when process variables exceed safe operating limits
Quality control reject defective products in real-time production lines
Cost reduction unplanned downtime costs orders of magnitude more than preventive action
Data integrity flag sensor malfunctions and corrupted measurements automatically

⚡ **Key insight** — in industrial systems, the cost of a missed anomaly *(false negative)* almost always exceeds the cost of a false alarm. Detection sensitivity must be tuned accordingly.

Three broad families of methods

Statistical, Machine Learning, and Rule-Based

Statistical Methods

Model the signal as a stochastic process and flag deviations beyond a threshold. Assumes a known (or learnable) underlying distribution.

Threshold-based fixed or adaptive bounds ($\mu \pm k\sigma$, IQR fences)
Moving average / EWMA track rolling mean and variance; alert on exceedance
CUSUM cumulative sum control chart; sensitive to small persistent shifts
Spectral methods FFT / PSD to detect unexpected frequency components

Machine Learning Methods

Learn a model of normality from historical data; anomalies are points the model cannot explain or reconstruct well.

Unsupervised Isolation Forest, One-Class SVM, k-NN density estimation
Reconstruction-based Autoencoder, VAE; high reconstruction error $\Rightarrow$ anomaly
Forecasting-based LSTM, Transformer predict next value; residual triggers alert

Signal Processing Methods

Exploit the time-frequency structure of the signal without explicit statistical assumptions.

FFT / STFT detect spurious harmonics or missing spectral peaks
Wavelet decomposition localise transient anomalies in both time and frequency
Hilbert–Huang transform non-stationary, non-linear signal analysis

⚡ **Practical choice** — signal processing methods excel at interpretability and low latency, making them ideal as a first detection layer in real-time embedded systems before heavier ML inference.

Fourier Analysis

Anomaly Detection in Sensor Data

4.3.1 Continuous Fourier Transform (CFT)

INFO

Any periodic *(or integrable)* signal $x(t)$ can be decomposed into a sum of sinusoids, each characterised by a frequency f, an amplitude, and a phase.

$$X(f) = \int_{-\infty}^{+\infty} x(t) \times e^{-2j\pi ft}\, \mathrm{d}t$$

$$x(t) = \int_{-\infty}^{+\infty} X(f) \times e^{+2j\pi ft}\, \mathrm{d}f$$

Properties

Linearity	$\alpha x + \beta y \to \alpha X + \beta Y$
Shift	$x(t - t_0) \to X(f)e^{-2j\pi f t_0}$
Convolution	$x(t) * h(t) \to X(f) \cdot H(f)$
Parseval	$\int \lvert x\rvert^2 \mathrm{d}t = \int \lvert X\rvert^2 \mathrm{d}f$

⚡ Unexpected frequency components or sudden spectral shifts betray faults in sensors and machinery.

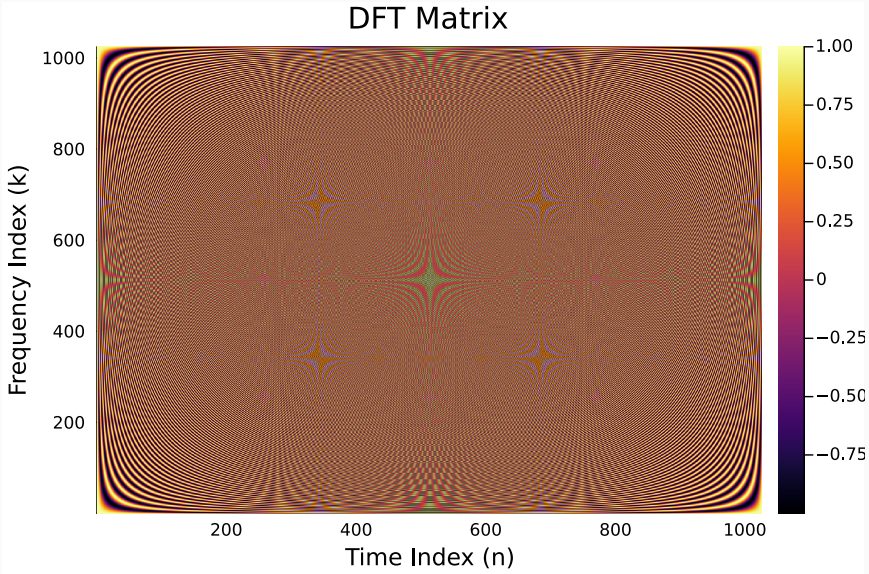
4.3.2 Discrete Fourier Transform (DFT)

Naïve DFT

For a sequence $x[n]$, $n = 0, \ldots, N-1$:

$$X[k] = \sum_{n=0}^{N-1} x[n] \times W_N^{kn} \quad \text{and} \quad x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] \times W_N^{-kn} \quad \text{where} \quad W_N = e^{-2j\frac{\pi}{N}}$$

```julia
N = 1024
I = 0:N-1
w = exp(-2 * pi * im / N)
DFT = w .^ (I' .* I)
using Plots
heatmap(real(DFT))
```

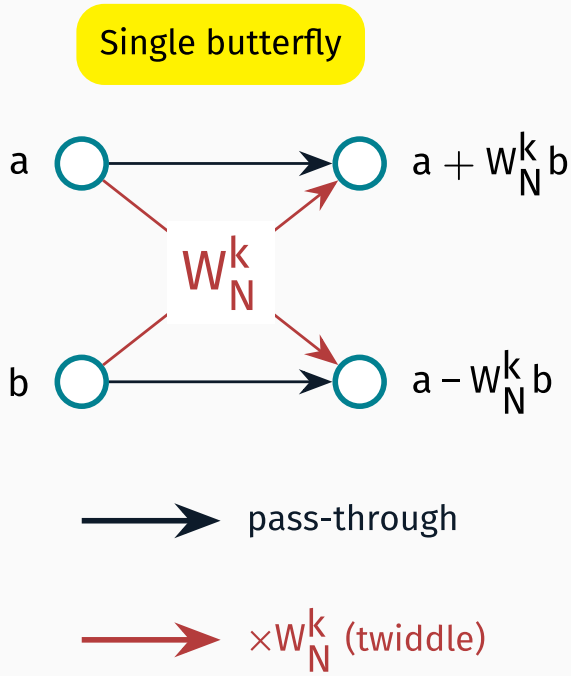


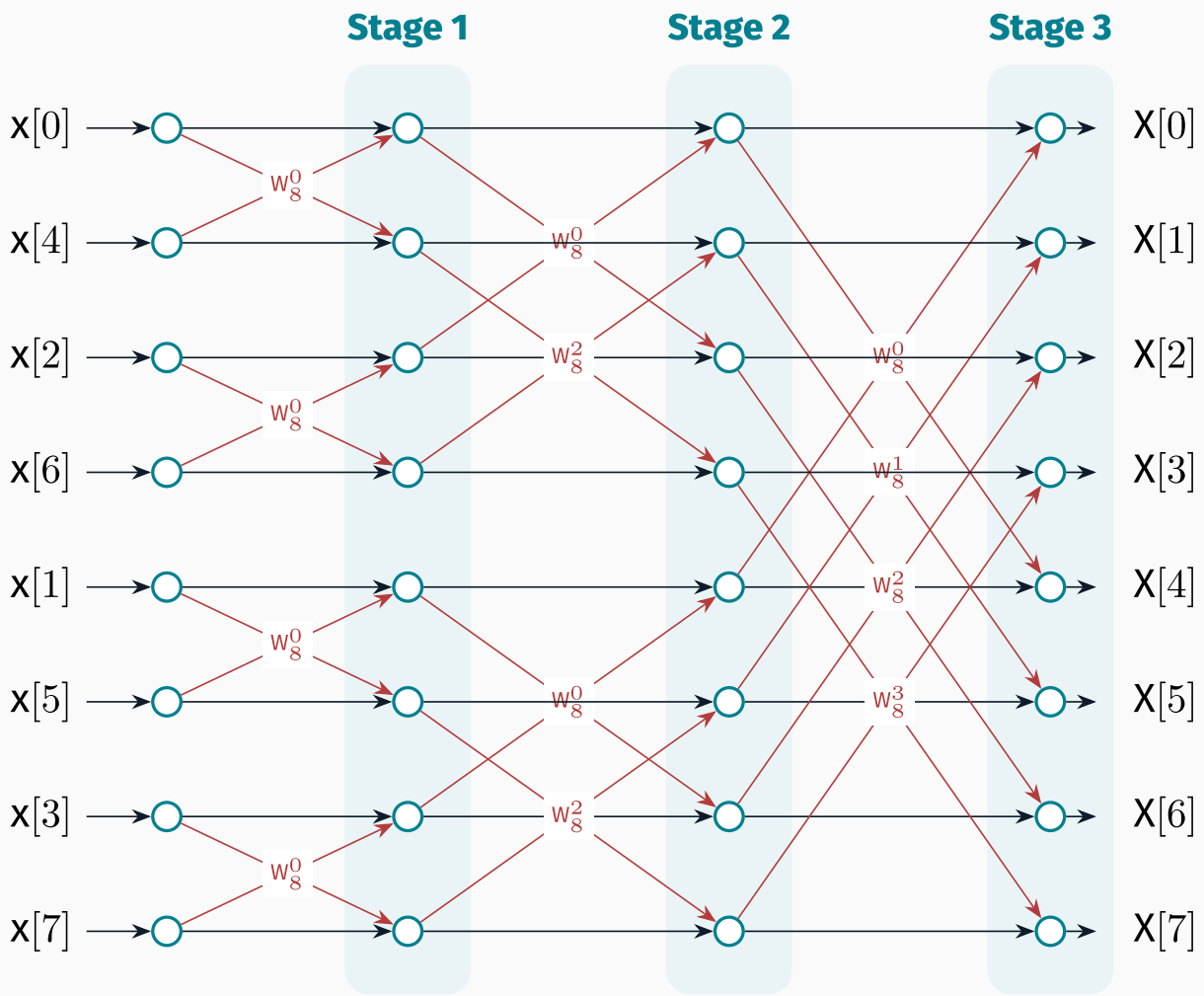
Cooley–Tukey *(divide-and-conquer)*

Split $N = 2^p$ into even / odd sub-sequences:

$$X[k] = \underbrace{\sum_{m=0}^{N/2-1} x[2m] \times W_{N/2}^{mk}}_{E[k]} + W_N^k \underbrace{\sum_{m=0}^{N/2-1} x[2m+1] \times W_{N/2}^{mk}}_{O[k]}$$

Butterfly recursion until $N = 1$.




Stage 1
Stage 2
Stage 3
x[0]
x[4]
x[2]
x[6]
x[1]
x[5]
x[3]
x[7]
X[0]
X[1]
X[2]
X[3]
X[4]
X[5]
X[6]
X[7]
W_8^0
W_8^0
W_8^0
W_8^0
W_8^2
W_8^0
W_8^2
W_8^0
W_8^0
W_8^1
W_8^2
W_8^3

Complexity

Algorithm	Cost
Naïve DFT	$\mathcal{O}(N^2)$
Cooley–Tukey FFT	$\mathcal{O}(N \log_2 N)$

Practical notes for streaming sensors

- Choose $N = 2^p$ *(e.g. 512, 1024)* for radix-2 FFT;
- Apply a window *(Hann, Hamming)* to reduce spectral leakage;
- Use overlapping frames (50 %) to track transient anomalies.

</> Source code is available at [codes/anomaly-detection-fft.jl]

4.3.3 Power Spectral Density (PSD)

The PSD is the Fourier transform of the autocorrelation function $R_x(\tau)$:

$$S_{xx}(f) = \int_{-\infty}^{+\infty} R_x(\tau) \times e^{-2j\pi f\tau} \, \mathrm{d}\tau$$

Equivalently:

$$S_{xx}(f) = \lim_{T\to\infty} \frac{1}{T} \left| X_T(f) \right|^2$$

Total power (Parseval)

$$P_x = \int_{-\infty}^{+\infty} S_{xx}(f) \, \mathrm{d}f = \overline{x^2(t)}$$

Estimation methods

Method	Description
Periodogram *(high variance)*	$\hat{S}(f) = \frac{1}{N}\lvert X[k]\rvert^2$
Welch *(low variance)*	Average periodograms over overlapping windows
Bartlett *(special case of Welch)*	Non-overlapping segments
AR / Burg *(high resolution for short records)*	Parametric

⚡ **Anomaly detection** — monitor baseline PSD; flag frames where spectral energy in a band exceeds a threshold γ *(e.g. $\mu + 3\sigma$ from a reference window)*.

</> Source code is available at [codes/anomaly-detection-psd.jl]

Thank you for your attention

Bibliography

Bibliography

Axelson, J. (2007). *Serial Port Complete: COM Ports, USB Virtual COM Ports, and Ports for Embedded Systems* (2nd ed.). Lakeview Research.

Barnett, R. H., Cox, S., & O'Cull, L. (2006). *Embedded C Programming and the Atmel AVR* (2nd ed.). Thomson Delmar Learning.

Campbell, J. (1984). *The RS-232 Solution*. Sybex.

Cooley, J. W., & Tukey, J. W. (1965). *An Algorithm for the Machine Calculation of Complex Fourier Series* (Vol. 19, Issue 90, pp. 297–301). American Mathematical Society. https://doi.org/10.1090/S0025-5718-1965-0178586-1

EIA. (1987,). *EIA-232-D: Interface Between Data Terminal Equipment and Data Circuit-Terminating Equipment*.

Forouzan, B. A. (2006). *Data Communications and Networking* (4th ed.). McGraw-Hill.

Horowitz, P., & Hill, W. (2015). *The Art of Electronics* (3rd ed.). Cambridge University Press.

IEEE. (2022,). *IEEE Standard for Ethernet*. https://standards.ieee.org/ieee/802.3/10422/

Kugelstadt, T. (2003). *The I2C-Bus Specification*. Texas Instruments.

Bibliography

Ledin, J. (2021). *Modern Embedded Computing*. Packt Publishing.

Leens, F. (2009a). *An Introduction to I2C and SPI Protocols* (Vol. 12, Issue 1, pp. 8–13). https://doi.org/10.1109/MIM.2009.4762946

Leens, F. (2009b). An Introduction to I2C and SPI Protocols. *IEEE Instrumentation & Measurement Magazine*, *12*(1), 8–13. https://doi.org/10.1109/MIM.2009.4762946

Maier, A., Sharp, A., & Vagapov, Y. (2014). Comparative Analysis and Practical Implementation of the SPI Bus. *Proceedings of the Internet Technologies and Applications (ITA)*, Compare. https://doi.org/10.1109/ITechA.2014.6956339

Motorola. (2000). *SPI Block Guide* (V03.06). https://web.mit.edu/6.115/www/document/spi.pdf

NXP Semiconductors. (2021). *I2C-bus Specification and User Manual* (No. UM10204; 7th ed.). https://www.nxp.com/docs/en/user-guide/UM10204.pdf

Oppenheim, A. V., & Schafer, R. W. (1999). *Discrete-Time Signal Processing* (3rd ed.). Prentice Hall.

Raspberry Pi Ltd. (2024,). *Getting Started — Raspberry Pi Documentation*. https://www.raspberrypi.com/documentation/computers/getting-started.html

Bibliography

Stallings, W. (2014). *Data and Computer Communications* (10th ed.). Pearson.

Stoica, P., & Moses, R. L. (2005). *Spectral Analysis of Signals*. Prentice Hall.

Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer Networks* (5th ed.). Pearson Prentice Hall.

Thomas, G. B. (1949). A Self-Clocking Telegraph Signal. *Post Office Electrical Engineers' Journal*, *42*, 141–143.

TIA. (1997,). *Interface Between Data Terminal Equipment and Data Circuit-Terminating Equipment Employing Serial Binary Data Interchange* (Issue TIA-232-F).

Valvano, J. W. (2014). *Embedded Systems: Introduction to ARM Cortex-M Microcontrollers* (5th ed.). Self-published.